

В цьому pdf документі знаходяться всі ті самі запити та їх результати скріншотами.

```
-- Завдання1: порахувати успішність студентів залежно від року навчання
-- Коментар: я інтерпретувала це так, що треба порахувати середню оцінку студента по всім предметам (у випадку, якщо у студента одна оцінка 76, а інша NULL, то буде  $(76 + 0) / 2$ , бо вважаємо, що підрахунок успішності проходить в кінці семестру, тобто NULL це 0), потім середню оцінку всіх студентів за певним навчальним роком. В результаті виводимо ім'я, прізвище студента, його навчальний рік, середню оцінку з усіх предметів та середню оцінку по його навчальному року (і для зручності посортувати)
```

```
SELECT
    s.name || ' ' || s.surname AS full_name,
    c.student_year AS study_year,
    ROUND(AVG(COALESCE(e.grade, 0)), 1) AS avg_student_grade,
    ROUND(AVG(AVG(COALESCE(e.grade, 0)))OVER (PARTITION BY c.student_year), 1)
AS avg_course_grade
FROM student s
    INNER JOIN enrolment e USING (student_id)
    INNER JOIN course c USING (course_id)
GROUP BY s.name, s.surname, c.student_year
ORDER BY study_year ASC, full_name ASC;
```

	full_name text	study_year smallint	avg_student_grade numeric	avg_course_grade numeric
1	Андрій Ковальчук	1	87.7	54.8
2	Ірина Біленко	1	0.0	54.8
3	Леся Павленко	1	93.5	54.8
4	Павло Демченко	1	38.0	54.8
5	Інна Корненко	2	93.0	64.4
6	Максим Новенко	2	43.5	64.4
7	Роман Беленко	2	42.0	64.4
8	Світлана Петрушен...	2	79.0	64.4
9	Анна Трофименко	3	88.0	52.6
10	Віталій Семенко	3	0.0	52.6
11	Катерина Миронен...	3	0.0	52.6
12	Михайло Костенко	3	83.0	52.6
13	Олег Гавриленко	3	92.0	52.6
14	Тамара Юрченко	4	0.0	0.0

-- Завдання2: для кожного з студентів знайти його середній бал у порівнянні з середнім балом по групі

-- Коментар: я це інтерпретувала, що треба знайти середній бал студента та середній бал групи і вивести ім'я, прізвище студента, назву групи, середній бал студента та групи

```
SELECT DISTINCT
    s.name || ' ' || s.surname as full_name,
    g.name as group_name,
    ROUND(AVG(e.grade) OVER (PARTITION BY s.student_id), 1) AS
avg_student_grade,
    ROUND(AVG(e.grade) OVER (PARTITION BY g.group_id), 1) AS avg_group_grade
FROM student s
    INNER JOIN enrolment e USING (student_id)
    INNER JOIN student_group g USING (group_id)
ORDER BY group_name ASC, full_name ASC;
```

	full_name text	group_name character (7)	avg_student_grade numeric	avg_group_grade numeric
1	Катерина Миронен...	БІ-14	[null]	92.0
2	Олег Гавриленко	БІ-14	92.0	92.0
3	Анна Трофименко	ЕК-04	88.0	88.0
4	Віталій Семенко	ЕК-04	[null]	88.0
5	Ірина Біленко	ІТ-12	[null]	76.0
6	Павло Демченко	ІТ-12	76.0	76.0
7	Андрій Ковальчук	КН-01	87.7	90.0
8	Леся Павленко	КН-01	93.5	90.0
9	Інна Корненко	МА-92	93.0	90.0
10	Максим Новенко	МА-92	87.0	90.0
11	Михайло Костенко	ПС-91	83.0	83.0
12	Тамара Юрченко	ПС-91	[null]	83.0
13	Роман Беленко	ФІ-23	84.0	81.5
14	Світлана Петрушен...	ФІ-23	79.0	81.5

-- Завдання3: порахувати статистику записів на курси для кожного року навчання: кількість курсів, кількість записів та кількість студентів, що вже отримали бали

-- Коментар: порахували для кожного навчального року (курсу) кількість курсів (предметів), кількість записів студентів на ці предмети та скільки студентів цих навчальних років мають 1 або більше оцінок, які не NULL

SELECT

```

        c.student_year AS course_year,
        COUNT (DISTINCT c.course_id) AS quantity_of_courses,
        COUNT (e.course_id) AS quantity_of_enrolment,
        COUNT(DISTINCT e.student_id) FILTER (WHERE e.grade IS NOT NULL) AS
students_with_marks
FROM course c
    LEFT JOIN enrolment e USING (course_id)
GROUP BY c.student_year
ORDER BY course_year ASC;

```

	course_year smallint	quantity_of_courses bigint	quantity_of_enrolment bigint	students_with_marks bigint
1	1	3	8	3
2	2	4	6	4
3	3	3	5	3
4	4	2	1	0

-- Завдання4: Для кожного курсу знайти в якому мінімальному семестрі він може читатись

-- Коментар: Інтерпретація: нам треба знайти курси та їх пререквізити. Враховуючи, що у нас є курси, які мають косвену залежність (курс А потребує курс В потребує курс С), то нам треба використовувати рекурсію для такого пошуку вглиб. у мене не вийшло повністю це зробити, але вирішила залишити приблизне щось, хоч воно і недопрацьовує

```

WITH RECURSIVE course_semester AS (
    SELECT
        c.course_id,
        c.name,
        1::INTEGER AS min_semester
    FROM course c
        LEFT JOIN course_prerequisite cp USING (course_id)
    WHERE cp.course_id IS NULL OR cp.prerequisite_course_id IS NULL
    UNION ALL
    SELECT
        c.course_id,
        c.name,
        cs.min_semester + 1 AS min_semester
    FROM course_prerequisite cp
        INNER JOIN course_semester cs ON cp.prerequisite_course_id =
cs.course_id
        INNER JOIN course c ON cp.course_id = c.course_id
)
SELECT
    course_id,
    name,
    MAX(min_semester) AS min_semester

```

```
FROM course_semester
GROUP BY course_id, name
ORDER BY min_semester, course_id;
```

	course_id integer	name character varying (32)	min_semester integer
1	1	Програмування	1
2	2	Математичний аналіз	1
3	3	Дискретна математика	1
4	6	Операційні системи	1
5	7	Архітектура комп'ютерів	1
6	12	Кібербезпека	1
7	4	Алгоритми та структури дан...	2
8	5	ООП	2
9	8	Веб-технології	2
10	10	Комп'ютерні мережі	2
11	9	Машинне навчання	3
12	11	Інженерія ПЗ	3

```
-- Завдання5: Знайти всіх студентів, які записані на більше курсів ніж в
середньому студенти
-- Коментар: Як я зрозуміла: спершу знаходимо кількість курсів (предметів) у
одного студента, далі рахуємо середню кількість курсів у всіх студентів. Далі
виводимо всіх, хто має кількість курсів більше за середню кількість.
```

```
WITH courses_per_student AS (
    SELECT student_id, COUNT(course_id) AS quantity_of_courses
    FROM enrolment
    GROUP BY student_id
),
avg_courses_per_student AS (
    SELECT AVG(quantity_of_courses) AS avg_count
    FROM courses_per_student
)
SELECT
    s.student_id,
    s.name || ' ' || s.surname AS full_name,
    cps.quantity_of_courses
```

```
FROM courses_per_student cps
  INNER JOIN student s USING(student_id)
  CROSS JOIN avg_courses_per_student acps
WHERE cps.quantity_of_courses > acps.avg_count;
```

	student_id [PK] integer 	full_name text 	quantity_of_courses bigint 
1	1	Андрій Ковальч...	3
2	2	Леся Павленко	2
3	3	Павло Демченко	2
4	5	Максим Новенко	2
5	7	Роман Беленко	2

-- Завдання6: Знайти топ-3 студенти у кожному курсі за отриманими балами
 -- Коментар: як я зрозуміла, то нам треба зробити рейтинг по
 курсам(предметам), в яких студенти з вищими оцінками будуть вище в топі і з
 нижчими нижче, а якщо у них оцінка null, то в топ вони не попадуть. топ може
 бути неповний. можна було зробити з корельованим підзапитом, але там виходило
 трохи повільніше.

```
WITH rank_list AS (
  SELECT
    e.course_id,
    e.student_id,
    s.name || ' ' || s.surname AS full_name,
    e.grade,
    ROW_NUMBER() OVER (PARTITION BY e.course_id ORDER BY e.grade DESC) AS
rank_of_student
  FROM enrolment e
    INNER JOIN student s USING (student_id)
  WHERE e.grade IS NOT NULL
)
SELECT course_id, rank_of_student, full_name, grade
FROM rank_list
WHERE rank_of_student <= 3
ORDER BY course_id, rank_of_student;
```

	course_id integer 	rank_of_student bigint 	full_name text 	grade smallint 
1	1	1	Леся Павленко	92
2	1	2	Андрій Ковальчук	85
3	2	1	Леся Павленко	95
4	2	2	Андрій Ковальчук	88
5	3	1	Андрій Ковальчук	90
6	3	2	Павло Демченко	76
7	4	1	Інна Корненко	93
8	5	1	Максим Новенко	87
9	6	1	Роман Беленко	84
10	6	2	Світлана Петрушен...	79
11	8	1	Анна Трофименко	88
12	9	1	Олег Гавриленко	92
13	10	1	Михайло Костенко	83